

A ‘watch your replay videos’ Reflection Assignment on Comparing Programming without versus with Generative AI: Learning about Programming, Critical AI Use and Limitations, and Reflection

Sarah “Magz” Fernandez
sarah.fernandez@maine.edu
University of Maine
Orono, Maine, USA

Greg L. Nelson
gregory.nelson@maine.edu
University of Maine
Orono, Maine, USA

Abstract

Generative AI is disrupting computing education. Most interventions focus on teaching GenAI use rather than helping students understand how AI changes their programming process. We designed and deployed a novel comparative video reflection assignment adapting the Describe, Examine, then Articulate Learning (DEAL) framework. In an introductory software engineering course, students recorded themselves programming during their team project two times: first without, then with using generative AI. Students then analyzed their own videos using a scaffolded set of reflection questions, including questions on their programming process and their help-seeking from humans, the internet, and AI. We conducted a qualitative thematic analysis of the reflections, finding students developed insights about planning, debugging, and help-seeking behaviors that transcended AI use. Students reported learning to slow down and understand before writing or generating code, recognized patterns in their problem-solving approaches, and articulated specific process improvements. Students also learned and reflected on AI limits and downsides, and strategies to use AI more critically, including better prompting but also to benefit their learning instead of just completing tasks. Unexpectedly, the comparative reflection also scaffolded reflection on programming not involving AI use, and even led to students spontaneously setting future goals to adopt video and other regular reflection. This work demonstrates structured reflection on programming session videos can develop metacognitive skills essential for programming with and without generative AI and also lifelong learning in our evolving field.

CCS Concepts

• Social and professional topics → Computing education.

Keywords

reflection, qualitative, generative AI, ChatGPT, video, programming

ACM Reference Format:

Sarah “Magz” Fernandez and Greg L. Nelson. 2026. A ‘watch your replay videos’ Reflection Assignment on Comparing Programming without versus with Generative AI: Learning about Programming, Critical AI Use

and Limitations, and Reflection. In *Proceedings of the 57th ACM Technical Symposium on Computer Science Education V.1 (SIGCSE TS 2026)*, February 18–21, 2026, St. Louis, MO, USA. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3770762.3772668>

1 Introduction

Learning to reflect is a part of being a reflective practitioner and a professional computer scientist, because reflection is an essential professional skill for improving over time. Reflection and iterative improvement are also a part of professional skills and widespread practices, such as agile software development, where sprint retrospectives serve as a form of reflection.

Generative AI is transforming how we program; this ongoing situation and need for learning makes reflection even more important, but our field knows relatively little about how to foster reflection on critical and ethical generative AI use specifically [10, 25, 26].

The education research literature has found that careful scaffolding of reflection [32, 34] greatly increases the quality of reflection and learning outcomes. The most common scaffolding is via a sequence of reflection steps, such as questions, which is called a script [37] in the literature (not the same as a computer program script). For example, the DEAL reflection framework involves three sequential steps: Describe, Examine, and Articulate Learning [3]. This framework is general and applicable to multiple domains, but must be implemented to fit the domain and subject of reflection. Scaffolding reflection includes designing the experience and emotional aspect, such as in usability testing, where students share their app and watch users try to use it, but without speaking or being able to do anything (which can be painful but very informative). Students observe directly and in detail where users struggle, spin their wheels, and misunderstand things, and they learn from it.

Drawing on those, we used and extended the DEAL framework to design a comparative video reflection activity, for doing a programming session without generative AI, then with generative AI. For the Describe step, we had students record and then later review a screen recording of each programming session. Next, for the Examine step, students reflected on challenges, progress, programming sub-activities such as planning, help and information seeking, writing, and debugging, then the differences between the sessions and potential causes of those differences. This general design idea of watching and comparing videos for reflection appears novel in computing education. For the Articulate Learning step, students synthesized their analyses, what they learned from that, how it was learned, why that learning matters, and what future goals they might set based on their learning.



This work is licensed under a Creative Commons Attribution 4.0 International License. *SIGCSE TS 2026*, St. Louis, MO, USA.

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2256-1/2026/02

<https://doi.org/10.1145/3770762.3772668>

We deployed this assignment in an undergraduate introductory software engineering class in Fall 2024, and thematically analyzed students' written submissions to answer:

- (1) **RQ1** When students reflect on their programming using videos and DEAL, what do they report learning related to a) programming while using generative AI tools, and b) programming in general?
- (2) **RQ2** When students reflect on their programming using videos and DEAL, what ideas for acting on their learning do they report in their reflection?

2 Related Work

There is good work on students reflecting on their programming, finding that it is generally positive; however, we have found no research on computing students reflecting using an assignment following the DEAL framework (a particular research-based reflection design framework). The DEAL reflection framework originated in the context of domain general research on service learning (such as doing a class project for a non-profit) [3, 27]. Within computing education, there is work on computing students reflecting on service learning with essays and journals over time, but not specifically scoped to their programming process or a couple programming sessions [7, 27]. Beyond that, many instructors do include some reflection in their courses but much of it is not published.

Much of the work in computing education focuses on reflection about comparing, evaluating, or explaining solutions to problems, rather than on reflecting on a problem solving process or programming session itself. This is often because the reflection is designed to accomplish worthy technical knowledge or narrow skill learning objectives; for example, for the learning objective "utilize the suitable map and reduce operations in Spark to transform and aggregate data" the reflection "Think of the solution you were attempting. Is this the same as what the bot presented in the chat? Take this time to explain the solution to each other" where the bot presented a solution that met the learning objectives [29, 31]. While that system involved a conversational agent to give hints and scaffold reflection of that kind, students did not reflect on their overall programming process, experience, or on using the agent itself [24, 30, 31]. Similarly, some work includes collaborative reflection and students comparing, extending, and challenging other students reasoning [1, 31] or pedagogical code review finding issues with code itself, not the process of coding [16]. Other work tries to scaffold metacognitive reflection at a more tactical level such as while solving a particular kind of problem [10, 17].

While some excellent work closer to ours has students reflect on their GenAI use [13, 21], only a few works have reflection on problem-solving process and GenAI use. But we are unaware of prior published work on assignments for students reflecting to compare their process when not using GenAI versus when using GenAI, nor comparing sessions. Some work has students reflect not on the programming process but instead comparing solutions, such as comparing one's code vs. code solely done by generative AI [10] e.g. "Compare the solutions and comment on the similarities and differences. Submit both solutions and your critique of the main differences between them." The closest work was research on understanding students; it did a brief content analysis of student

responses to an optional reflection question after an assignment on describing their process to complete the assignment, when and why they used AI, and how it was helpful or not [23]. Lyu et al analyzed some student open-ended reflections, and their course included using GenAI, pairing with a human, or both, but there is no description of the reflection questions or topics [21]. Long before GenAI but also in response to GenAI, work has called for increasing integration of metacognition in computing, including "critical thinking and reflection" [6, 10, 20, 33].

While video reviewing is common in qualitative research and for coaching [11, 19], having students review videos of their work as an assignment is rare in computing education, and comparing sessions appears novel. Students have recorded retrospective video explanations of their solutions, such as on assignments [36] or as "post-exam videos" [35], like an oral exam where they explain and correct mistakes as they explain their solutions. Some works study video reflection for teacher training [2, 5].

There is work on reflecting on code editor replays (keystroke level) on small programming problems, but not videos which also show more information seeking and other behaviors on the screen. We are aware of only two works. In 2021 Ditton et al piloted editor replays finding some benefits and student frustrations around value of reviewing them [8]. In 2023 Xie et al found students who reflected on code replays exhibited better self-regulation, such as planning more often [38]. There are also research studies where the researchers analyze student programming sessions while using generative AI with video [26], which involves students thinking out loud or reflecting in a diary or survey question [23], but not as an assignment.

3 Design of reflection assignment

We deployed this reflection assignment in an undergraduate software engineering class, delivered synchronous fully online, in Fall 2024 at University of Maine Orono, a northeastern rural university. For context, generative AI use was covered explicitly in this course, including basic prompt engineering with roles and output formatting, practice using GenAI for feedback on requirements, a class session where a TA demoed using Cursor on their personal website project, and a class session on generating tests using GenAI. The students recorded programming sessions in the context of working on a software engineering team project, with 7 of 8 projects involving web development using React and Firebase (one was a Discord bot). The screen recorded videos of their programming sessions were recorded by students before the assignment was released, though they were told they would be used for a "watch the replay" reflection assignment. Students were reminded to, when recording, to only record their screen, turn off their notifications, and edit out of their videos anything they might feel uncomfortable sharing.

We applied the DEAL framework, which is a high level architectural pattern for designing reflection. The sequential steps in DEAL are Describe, Examine, and Articulate Learning [3]. Normally each step has a set of reflection questions to respond to, usually in writing. We detail our reflection assignment below ([source files](#)).

In our design for DEAL reflection, the Describe step was a) work on their team software project while recording a video of a session without using GenAI, then a session with using GenAI, then b)

later review each video by watching it. The instructions were to record a programming session while working on their class project (programming they would do anyways) during their second to last project milestone/sprint, then record another one while working on their final milestone/sprint. In contrast, without videos, traditionally in DEAL the Describe step involves writing long descriptions of what happened, focusing on facts and observations [3].

In the Examine step, students answered a set of written reflection questions in the following order. To answer these, the assignment had students fill in a table with four columns with headers "Reflection question", "Answer for Session 1", "Answer for Session 2", "Answer for Compare the differences". Each row had one reflection question, for example, "What were you working on?". First, students were instructed to watch video 1 then answer all of the Examine questions, then repeat that for video 2 (we will describe the "compare the differences" column in the next paragraph). The sequence of questions started with questions to make sense of what happened at a higher level: 1) What were you working on? 2) What progress was made? 3) If you did pair programming, who did you pair with?, 4) What resources, people, or tools did you work with? 5) What challenges did you encounter?, and 6) How did you respond to each challenge? Next, students answered detailed questions on roughly what percent of their time was spent in different categories while programming, related to basic steps in the programming process. The first three time categories were: planning, writing or editing code manually, debugging or understanding code. Next were three time categories related to information seeking and human collaboration: looking up and using examples, looking up and using API documentation, asking another person for help, and working with another person. Finally, there were time categories for using generative AI: using generative AI to write code, using generative AI to understand code or ask questions about APIs, etc, using generative AI to debug code, using generative AI for planning.

In the Examine step, students also compared their two programming sessions, without generative AI, and with generative AI. First, they answered what could be improved about each session, and what they did differently for the second session (usually just using generative AI). For each reflection question from the prior steps, students answered "Compare and Evaluate: write about 3-5 bullet points total per question on a) the differences between the two videos / programming sessions, b) what caused those differences." Those answers were written in the last column of the reflection questions table, each in a cell with "differences:", some whitespace, then "causes of differences:".

Next, for the Articulate Learning step, students answered the following questions. For context, video 1 was the session without using generative AI, and video 2 was the session with using generative AI. The last five questions are from the DEAL framework examples in [3], with the last question broadened:

- (1) Summarize the most significant differences between your software development session in Video 1 vs. Video 2
- (2) Summarize the causes of those differences and what their effects/consequences were
- (3) What did I learn?
- (4) How, specifically, did I learn it?
- (5) Why does this learning matter, why is it important?

- (6) In what ways will I use this learning?
- (7) What goals might I set in accordance with what I have learned in order to improve myself and/or the quality of my learning and/or the quality of my future?

The assignment instructions also included instructions to scope the reflection and help students manage their time, with time guidelines for each video, and explicit length norms for each question, to make the assignment manageable for students. For nearly all of the questions, the instructions were to "Write about 3-5 bullet points per question" (except the two articulate learning questions on summarizing the most significant differences between the videos, and the causes and effects, which asked for a paragraph each). When watching the video, we instructed students to be efficient i.e., "watch each video at 2x or higher speed, click through the video, etc."

4 Methods

Thirty-nine students enrolled in the course, with 36 active in the course in the last three weeks of the semester. No students opted out of the research. This research was reviewed by our institutional ethics board and approved. Twenty-eight (93%) of 30 responded to a course demographics survey. Of the 28 respondents, 19 (68%) identified as men, 8 (29%) as women, and one (4%) as non-binary. They included 14 (50%) 3rd year students, 8 (29%) 4th year, and 2 (7%) 2nd year students, one (4%) 6th year, and 3 (14%) graduate students. Twelve (42%) reported a disability or mental health condition, with 9 (32%) Attention Deficit, 5 (18%) a "Mental health condition", 2 (7%) a Learning disability, 1 (4%) Autism, 1 (4%) Deaf or hard of hearing, and 1 (4%) PTSD. Beyond that, three others responded unsure or preferred not to answer. Twenty (71%) identified as White or Caucasian, 2 (7%) Asian, two (7%) Black or African American, 1 (4%) Hispanic or Latinx, and one (4%) preferred not to disclose. Ages mostly ranged from 19 to 23 years (21 respondents), with two (7%) 24-year-olds, one (4%) 25, and four (14%) between 28 and 33.

At our institution, prior to this course, students generally had not done group projects, and pair programming is not taught in the curriculum. Four out of thirty students chose to do their second session with GenAI and human pair-programming. While a packet on pair programming was distributed, students did not receive explicit instruction on pair programming itself, but did experience virtual pair programming and set up VSCode for that in the 75 minute class exercise on test-driven development, including using GenAI for generating tests.

For our analysis, a person unaffiliated with the university and the course instructor conducted a qualitative thematic analysis [4, 15] of the student's written reflection submissions. We first wrote memos on student whole answers to each question, reading each student as a whole, which we then discussed. We then decided to scope our analysis to specific questions. We then created a large affinity diagram of their responses and clustered them to identify themes, using the students' bullet point formatted responses. We chose questions from the "articulate learning step" of their reflections for our analysis: "What did I learn?" and "In what ways will I use this learning?", then "What goals might I set in accordance with what I have learned in order to improve myself and/or the quality of my learning and/or the quality of my future?". We chose these questions to balance a manageable set of data to analyze while also

still covering what students learned, as well as how students may use their learning and the future goals they might set, which helps gauge supporting their reflection to be actionable for putting that learning into practice.

Throughout our analysis, rather than calculating inter-rater reliability, we prioritized resolving disagreements through collaborative discussion to deepen shared understanding, consistent with practices for qualitative rigor in computing education [9, 15, 28]. We came to agreement on and chose to report themes to cover important patterns, including any potential harms, in the data. Each theme had at least 3 unique students. The reflection questions were answered by students without any example answers (besides a few generic ones like "Spent a lot of time on <activity> because <why>"). The assignment was graded for completion only.

5 Results

As noted in our method, we are reporting **themes**, not asserting specific numbers of students experienced them, but instead broadly reporting the diverse kinds of learning and goals set by students.

5.1 RQ1a: Learning related to programming with generative AI

AI Downsides and Negative Impacts on Learning: Students reported learning about downsides and negative impacts from using GenAI. For example, P32: "I also learned that using Generative AI causes me to learn less than actually coding manually.", and P10: "I learned that ai can be helpful when used in the right way, but it can also be unreliable, and I can see how it could have negative effects on my learning". Other students learned that over-reliance can lead to problems, like P20: "Relying too heavily on generative AI leads to different problems", and other students learned AI use can be situationally counter-productive, such as P11: "Generative AI can sometimes be extremely counterproductive, especially when used on an already strange piece of code." Another student, P14, commented on how writing code from scratch themselves can be more productive than using code from GenAI or the internet.

AI Limitations: Students reported learning about limitations of the GenAI tools. For example, P18: "I learned that AI for code generation has flaws.", P13: "Sometimes Gemini's summaries and reading comprehension suck.", and P26: "Github Co-Pilot isn't as good at creating tests." This also included reinforcing prior learning, like P19: "I reinforced the understanding I already have that when not used carefully AI often creates a lot of problems with junk code or in the case of the first video doesn't know enough context and will lead you down rabbit holes for simple solutions."

AI for Debugging and Critical Use of AI for Debugging: Students reported learning around debugging skills. This included that AI was useful for debugging, such as P21: "...[AI] can explain the issues well too.", but also included critical use and non-use of AI for debugging, for example, P1: "How to use AI more effectively in debugging", P8: "Manual debugging and understanding code is key whether using AI or not.", and P11: "I continue to be best at fixing bugs when I do so by reading examples or documentation."

AI Prompt Engineering: Students reported learning skills around prompt engineering. For example, student reflections included P8: "Prompting AI effectively is important to get the results

you want.", P20: "Effective communication with AI tools is critical to getting useful outputs.", and P14: "How [AI] is a helpful resource but can't fully help you write code due to it missing some context."

AI Utility and Benefits: Students reported learning about the utility and benefits of Generative AI in programming tasks. For example, student reflections included P21: "AI is actually very good at specific coding problems, and it can explain the issues well too.", P4: "I learned that if used correctly GenAI can be a useful tool.", and P30: "AI tools are really powerful and can help me streamline certain processes like writing boilerplate code and comments."

AI Increases Productivity: Students specifically reflected that Generative AI increased their productivity and efficiency. For example, P15: "Generative AI can significantly enhance productivity when used effectively.", P3: "...GenAI can save time.", and P8: "GenAI is super useful for debugging and faster coding."

5.2 RQ1b: Learning related to programming in general

Need for Background and Understanding Students reported learning the importance of having a solid background and understanding of tools, APIs, and their project codebases before implementation. For example, student reflections included P30: "I had no chance of coding manually in the first video, because I lacked the understanding", P8: "Manual debugging and understanding code is key whether using AI or not.", and P1: "Researching libraries' feasibility ahead of time before implementing them."

Planning: Students reported learning the value of planning. For example, student reflections included P27: "I learned that planning ahead of time is incredibly important...", P20: "Planning and structuring work before starting can help reduce issues later.", and P30: "I learned that having a plan on what to do really helps. When I tried random code bits aimlessly, I lost a lot of time."

Human help seeking: Students reported learning to seek help from people. For example, P27: "...it is sometimes way more efficient to just ask for help.", P17: "I learned how to pair program and ask for help and be vulnerable when I was confused and needed help from others." At least one student learned that human help seeking was preferable to AI, for example, P32: "I learned that asking a person sometimes is better than asking AI."

Metacognitive insights about learning: Students reported positive metacognitive insights about how they learn programming. For example, student reflections included P30: "It's okay to start a feature even when you don't know what to do. You try a few things, and start building your knowledge base. And you try to learn each time you fail.", P5: "If a task is taking a while and going nowhere, move on to something else.", and P13: "Learning the API is important, but this can't be done (easily) without coding."

Self-learning and professional identity Students reflected learning more about themselves and developing professional identity. For example, P21: "I won't do test driven development (At least not that way). I'm a game dev, it's called playtesting", P10: "I learned about myself as a programmer", and P13: "This learning will be used in a never-ending struggle for self-learning and improvement."

Specific technical learning Students identified specific skills they learned in addition to GenAI. Examples include student reflections such as P17: "I learned how to use css to get my UI to look a

specific way.", P29: "Learned how to use .md files.", and P16: "How the server receives request."

5.3 RQ2: Student-written goals for acting on their learning from their reflection

Students in the reflection generated very diverse goals for acting on their learning, including more skillful and critical GenAI use, but just as often on improving programming generally with better help-seeking, human connection and collaboration, and even purposefully not coding with GenAI.

Balance in using GenAI: Students set the goal of achieving a balance between AI use and other skills. For example, P8: "Balance using AI with traditional tools so i keep my core programming skills while also working faster.", P20: "Develop better planning and organizational skills to reduce reliance on AI and reactive problem-solving.", and P1: "Maintain a balance between manual problem-solving and using AI to debug problems."

Better Prompt Engineering: Students set the goal to learn better prompt engineering. Examples include P20: "Set a goal to allocate time for improving communication with AI tools, such as learning advanced prompting techniques.", P15: "Focus on learning more about efficient prompt engineering and API usage to streamline future projects.", P24: "I will set a goal to improve my AI prompting and documentation research."

More critical use of AI: Students set the goal of being more critical in their use of AI. For example, P8: "Build a habit of reading through AI generated code to understand the logic and continue learning my skills past school.", and P20: "Enhance my ability to debug and refine AI code by deepening my understanding of the underlying concepts.". This also included hesitant students opening up to more critical AI use, for example, P4: "I want to set the goal of being more open minded when it comes to certain uses of AI. I generally dislike it, especially when it comes to anything creative. But using it to direct some code, or give a suggestion on how to refactor something is useful, and doesn't impact my creativity or harm others."

Try out using AI more: Students set general goals of just trying to use AI more. For example, P32: "I will be more open to using AI, instead of having a sense of guilt when using it to write programs.", P19: "Become more effective at using AI generated code in a clean and concise way that makes my coding more efficient instead of less.", P15: "Develop a consistent approach to using generative AI for debugging, understanding APIs, and optimizing code."

Adopting a practice of regular self-reflection: Students set the goal of more self-reflection. Examples of self-reflection related goals include P10: "Goals I will set for myself: Keep self reflecting and checking my progress as a student and a programmer.", P18: "I'd like to go back and review my old code every once in a while to see my improvement.", and P25: "Sometimes I should try changing something a little bit in a way I approach a particular activity to eventually optimize it. The week after finals I am going to think of such activities and then for 30 days notice them and try to improve."

Adopting a practice of regular self-reflection with videos: Students included goals around reflecting on video recordings of themselves. Examples included P32: "I will analyze my work in the future through video in time intervals (one month apart) to see how I have improved as a programmer.", P31: "watching myself work

month to month to see what im struggling on.", and P5: "Maybe record my work again in the future."

Collaborate with other humans: Students set the goal to collaborate with others. Examples include, P27: "I will set goals such as, reviewing my code with a peer or mentor, this way I can receive the help i need ahead of time rather than being stubborn and wanting to finish the project on my own.", P31: "do more pair programming to see better ways other people do things.", and P8: "Actively look for more opportunities to collaborate with others on programming."

Ask for help sooner instead of struggling: Students set the goal to ask for help from others. For example, P17: "Set a time limit when I am stuck on something before I reach out for help so I don't spend a significant amount of time trying to understand something.", P16: "ASK MORE QUESTIONS!!! Write down questions and ask them.", and P22: "don't sit around and struggle on things by yourself ask for help sooner."

Coding without AI: Students set the goal to learn coding without relying on AI or for comparative experimentation. Examples include P18: "I'd like to practice coding without AI, to guarantee that I have a strong foundation, regardless of what technology there is.", P30: "I might save the day with the code I borrowed on the internet, or the code that AI wrote but there will be a day when I need to expand on my project...It is not a good habit to write code that I don't understand." This also included self-experimenting, such as P32: "I will work on two different projects, one using no generative AI, and using a lot of generative AI in the other one. I want to do this to test out how effective Gen AI is, how efficient it is, and how much I learn from using no AI to using AI."

Understanding coding tools and fundamentals: Students set the goal to learn coding tools and fundamentals; for example, P19: "Learn more about coding and file fundamentals", P31: "make sure i know my tools well before starting", and P11: "In the future, I can do exactly what I said above: start with documentation, and increase the efficiency of my programming." Students also mentioned learning then making artifacts to help them, such as P17: "Have a list of resources I can use to solve errors or issues that I run into". This sometimes included using GenAI to aid learning, such as P25: "I am going to start using generative AI more for learning. I am going to ask more follow-up questions to better understand a particular topic..."

Metacognitive and process goals: Students set goals related to metacognitive skills. Examples include P15: "Develop a consistent approach to using generative AI for debugging, understanding APIs, and optimizing code.", P10: "I won't be afraid to use new tools to help myself, and I will know when to recognize what tools might be a hindrance to my learning.", and P23: "Learn how to fix mistakes in the coding process."

Specific/Actionable Planning: Students set goals around the importance of planning, including specific and actionable goals, such as P15: "Allocate a fixed amount of time for planning and research at the start of every project.", P16: "Spend at least an hour planning before digging into writing code.", and P6: "Set 30 minutes aside for me to plan before I start the actual session."

Breaking down tasks: Students set the goal of breaking down larger tasks better. Examples include P6: "Break my work into chunks to limit the amount of time I might get distracted partway

through a work session.", P11: "Write down my tasks somewhere and keep them up while I program.", and P29: "Work in chunks."

Time Management: Students set goals related to time management strategies. Examples include P1: "Keep track of time spent on something to understand if it is wasting more time than needed.", P5: "Focus on timeliness in my future coding sessions.", and P23: "Code for longer periods of time in one sitting."

6 Limitations

Students did the reflection as part of a final assignment in the class due during finals. The timing of the assignment may also have reduced the depth of the student reflections, as they worked during a busy time. Students without submissions had final grades of 1 C grade, 2 B-, 1 B+, 1 A-. Excluding the reflection assignment points, students turning in the reflection had an average course grade of 87.7 versus 83.9 for those who did not submit. The above excludes three students who already stopped attending and doing course assignments, specifically the last project milestone, before the reflection was assigned.

We limit our claims and results to the course and student population in our dataset. Given the novelty of our reflection design, it can likely be improved and potentially personalized for different student populations or course contexts. Different or more scaffolding might be required particularly for more novice computing students. Recall and/or social desirability bias may have influenced student responses; our assignment was graded for completion only and was the last assignment of the semester, which reduces that bias [22], as does using video to ground the reflection [14]. For context, about 15% of active students just did not submit the assignment.

7 Discussion

We contribute a novel design for student reflection on their programming practices using comparative video analysis. Our approach also asks students to record themselves programming both with and without generative AI tools, then compare these sessions. Critically, we also used the DEAL framework [3] to design scaffolding for reflection, providing students with guided questions for describing, examining, and articulating learning from their experiences. Our results showed that this reflection assignment supported students' learning about generative AI, including its downsides, negative impacts, and limitations, while also supporting reflection on more critical usage patterns and prompt engineering skills. Students also demonstrated learning about programming in general, such as the need for background understanding before coding, planning, human help-seeking, meta-cognitive insights about their learning processes, and even professional identity development.

Our work demonstrates how video-based comparative reflection can broadly deepen learning in computing education, including for more critical GenAI use. Students noticed patterns and trade-offs in their behaviors, learned, and developed concrete goals for improvement. As computing education grapples with integrating GenAI [23, 26], scaffolded comparative reflection can help fill the wide gap identified in prior work [25] on how to help students critically reflect on then improve their GenAI use practices. This first version of our method even appears to provide a scalable way to help students develop more meta-cognitive awareness not only

about their GenAI tool use, but also programming in general, and their learning processes, even without external faculty feedback (adding that would definitely help even more, and more research is needed). Completely unexpected, students even set goals to reflect more in the future. Regular reflective habits are key for lifelong learning throughout their careers as the tech landscape continues to evolve, especially given GenAI.

In our analysis we saw only a few or no students, when they did our carefully scaffolded reflection, who appeared to be threatened by GenAI or give GenAI too much credence; they thought about GenAI as something to use critically with pros and cons and to be used responsibly. When scaffolded, the quality of the student reflection was often very high, and engaged with critical and ethical use of AI without being prompted to do so in the reflection questions themselves. When we properly scaffold and support their learning, at least in upper-level classes, maybe we can engage with our students as partners in navigating generative AI. Future work and instructors should engage in exploring and refining scaffolded reflection in lower-level courses, where results might be different or more scaffolding may be need to be designed.

Directly inspired by our results, and using our video-based reflection approach, we suggest instructors and researchers design and facilitate student self-experimentation by working on one project without GenAI and another with GenAI to test, as P32 said "how effective Gen AI is, how efficient it is, and how much I learn from using no AI [versus] using AI." Within that, and for future work on reflection and students acting on reflection longitudinally, we recommend follow-up with students with a survey and interviews to see how well they retain and act on their learning and goals they set from reflection. We also propose randomized quantitative evaluation of scaffolded comparative video reflection, such as during a course instead of just the end of a course.

Inspired by our specific comparative video reflection design, we think there is a large and fruitful design space for student reflection and learning, based on adapting qualitative research techniques. For example, having research participants review and reflect on videos is actually a qualitative research technique called video-stimulated recall [12, 18, 39]. Designers might simplify different qualitative research method practices with respect to replicability and rigor, for example, with new tools like a video reflection annotation tool [19]. But the design space is much broader. Ponder your favorite research studies; our work was inspired by Prather et al's "widening gap" study on novice GenAI use [26]. Fundamentally, constructing knowledge happens in qualitative research, and students need to construct knowledge also.

Maybe we can even blur the line between reflective learning and research to, for example, more rapidly figure out what skills for good GenAI use are, then teach them [25]? What if students or professionals using GenAI in industry recorded themselves while building more complex programming projects, then did scaffolded reflection on the skills and knowledge they used, and how those skills interacted with GenAI helping or hindering their progress and learning? Everyone might benefit. Researchers, industry professionals, instructors, and even students might do qualitative analysis of videos to help identify key skills and skill-building moments, and define more useful, critical, and ethical GenAI-integrated curricula.

References

- [1] David Adamson, Gregory Dyke, Hyeju Jang, and Carolyn Penstein Rosé. 2014. Towards an Agile Approach to Adapting Dynamic Collaboration Support to Student Needs. *International Journal of Artificial Intelligence in Education* 24, 1 (Jan. 2014), 92–124. <https://doi.org/10.1007/s40593-013-0012-6>
- [2] Jordan Allison. 2023. Classifying the Characteristics of Effective Continuing Professional Development (CPD) for Computer Science Teachers in the 16-18 Sector. *ACM Trans. Comput. Educ.* 23, 2 (June 2023), 26:1–26:30.
- [3] Sarah L Ash and Clayton Patti. 2009. Generating, Deepening, and Documenting Learning: The Power of Critical Reflection in Applied Learning. *Journal of Applied Learning in Higher Education* 1 (Jan. 2009), 25–48.
- [4] Virginia Braun and Victoria Clarke. 2006. Using Thematic Analysis in Psychology. *Qualitative Research in Psychology* 3, 2 (Jan. 2006), 77–101.
- [5] Cassandra Broneak, Chery Lucarelli, and Jennifer Rosato. 2019. Exploring the Use of Video Reflection as a Professional Development Tool. In *Proceedings of the 2019 ACM Conference on International Computing Education Research (ICER '19)*. Association for Computing Machinery, New York, NY, USA, 293.
- [6] Ünal Çakıroğlu and Betül Er. 2023. A Model to Develop Activities for Teaching Programming through Metacognitive Strategies. *Thinking Skills and Creativity* 48 (June 2023), 101279. <https://doi.org/10.1016/j.tsc.2023.101279>
- [7] Jörn Christiansson, Erik Grönvall, and Signe Louise Yndegren. 2018. Teaching Participatory Design Using Live Projects: Critical Reflections and Lessons Learnt. In *Proceedings of the 15th Participatory Design Conference: Full Papers - Volume 1 (PDC '18)*. Association for Computing Machinery, New York, NY, USA, 1–11.
- [8] Joseph Ditton, Hillary Swanson, and John Edwards. 2021. External Imagery in Computer Programming. In *Proceedings of the 52nd ACM Technical Symposium on Computer Science Education (SIGCSE '21)*. Association for Computing Machinery, New York, NY, USA, 1226–1231. <https://doi.org/10.1145/3408877.3432426>
- [9] Jayne Everson, F. Megumi Kivuva, and Amy J. Ko. 2022. "A Key to Reducing Inequities in Like, AI, Is by Reducing Inequities Everywhere First": Emerging Critical Consciousness in a Co-Constructed Secondary CS Classroom. In *Proceedings of the 53rd ACM Technical Symposium on Computer Science Education*. ACM, Providence RI USA, 209–215. <https://doi.org/10.1145/3478431.3499395>
- [10] Tony Haoran Feng, Andrew Luxton-Reilly, Burkhard C. Wünsche, and Paul Denny. 2024. From Automation to Cognition: Redefining the Roles of Educators and Generative AI in Computing Education. [arXiv:cs/2412.11419](https://arxiv.org/abs/2412.11419)
- [11] Ruben Georges Fukkink, Noortje Trienekens, and Lisa J. C. Kramer. 2011. Video Feedback in Education and Training: Putting Learning in the Picture. *Educational Psychology Review* 23, 1 (March 2011), 45–63.
- [12] Vince Geiger, Tracey Muir, and Janeen Lamb. 2016. Video-Stimulated Recall as a Catalyst for Teacher Professional Learning. *Journal of Mathematics Teacher Education* 19, 5 (Oct. 2016), 457–475. <https://doi.org/10.1007/s10857-015-9306-y>
- [13] Virginia Grande, Natalie Kiesler, and Maria Andreina Francisco R. 2024. Student Perspectives on Using a Large Language Model (LLM) for an Assignment on Professional Ethics. In *Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1 (ITICSE 2024)*. Association for Computing Machinery, New York, NY, USA, 478–484. <https://doi.org/10.1145/3649217.3653624>
- [14] Christine Hamel, Anabelle Viau-Guay, and Bernard Nkuyubwatsi. 2019. Using Video to Support Teachers' Reflective Practice : A Literature Review. *Cogent Education* 6, 1 (Jan. 2019), 1673689. <https://doi.org/10.1080/2331186X.2019.1673689>
- [15] David Hammer and Leema K. Berland. 2014. Confusing Claims for Data: A Critique of Common Practices for Presenting Qualitative Research on Learning. *Journal of the Learning Sciences* 23, 1 (2014), 37–46.
- [16] Christopher D. Hundhausen, Anukrati Agrawal, and Pawan Agarwal. 2013. Talking about Code: Integrating Pedagogical Code Reviews into Early Computing Courses. *ACM Trans. Comput. Educ.* 13, 3 (Aug. 2013), 14:1–14:28.
- [17] Majeed Kazemitabaar, Oliver Huang, Sangho Suh, Austin Z Henley, and Tovi Grossman. 2025. Exploring the Design Space of Cognitive Engagement Techniques with AI-Generated Code for Enhanced Learning. In *Proceedings of the 30th International Conference on Intelligent User Interfaces (IUI '25)*. Association for Computing Machinery, New York, NY, USA, 695–714.
- [18] Udo Kuckartz. 2014. *Qualitative Text Analysis: A Guide to Methods, Practice & Using Software*. SAGE Publications Ltd, 1 Oliver's Yard, 55 City Road, London EC1Y 1SP United Kingdom. <https://doi.org/10.4135/9781446288719>
- [19] Kim Chau Leung and Mei Po Shek. 2021. Adoption of Video Annotation Tool in Enhancing Students' Reflective Ability Level and Communication Competence. *Coaching: An International Journal of Theory, Research and Practice* 14, 2 (July 2021), 151–161. <https://doi.org/10.1080/17521882.2021.1879187>
- [20] Dastyni Loksa, Lauren Margulieux, Brett A. Becker, Michelle Craig, Paul Denny, Raymond Pettit, and James Prather. 2022. Metacognition and Self-Regulation in Programming Education: Theories and Exemplars of Use. *ACM Trans. Comput. Educ.* 22, 4 (Sept. 2022), 39:1–39:31. <https://doi.org/10.1145/3487050>
- [21] Wenhan Lyu, Yimeng Wang, Yifan Sun, and Yixuan Zhang. 2025. Will Your Next Pair Programming Partner Be Human? An Empirical Evaluation of Generative AI as a Collaborative Teammate in a Semester-Long Classroom Setting. [arXiv:cs/2505.08119](https://arxiv.org/abs/2505.08119)
- [22] Stephen Maloney, Joanna Hong-Meng Tai, Kristin Lo, Elizabeth Molloy, and Dragan Ilic. 2013. Honesty in Critically Reflective Essays: An Analysis of Student Practice. *Advances in Health Sciences Education* 18, 4 (Oct. 2013), 617–626.
- [23] Lauren E. Margulieux, James Prather, Brent N. Reeves, Brett A. Becker, Gozde Cetin Uzun, Dastyni Loksa, Juho Leinonen, and Paul Denny. 2024. Self-Regulation, Self-Efficacy, and Fear of Failure Interactions with How Novices Use LLMs to Solve Programming Problems. In *Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1*. ACM, Milan Italy, 276–282.
- [24] Atharva Naik, Jessica Ruhan Yin, Anusha Kamath, Qianou Ma, Sherry Tong-shuang Wu, Charles Murray, Christopher Bogart, Majd Sakr, and Carolyn P. Rose. 2024. Generating Situated Reflection Triggers about Alternative Solution Paths: A Case Study of Generative AI for Computer-Supported Collaborative Learning. <https://doi.org/10.48550/arXiv.2404.18262> [arXiv:cs/2404.18262](https://arxiv.org/abs/2404.18262)
- [25] James Prather, Juho Leinonen, Natalie Kiesler, Jamie Gorson Benario, Sam Lau, Stephen MacNeil, Narges Norouzi, Simone Opel, Vee Pettit, Leo Porter, Brent N. Reeves, Jaromir Savelka, David H. Smith, Sven Strickroth, and Daniel Zingaro. 2025. Beyond the Hype: A Comprehensive Review of Current Trends in Generative AI Research, Teaching Practices, and Tools. In *2024 Working Group Reports on Innovation and Technology in Computer Science Education (ITICSE 2024)*. Association for Computing Machinery, New York, NY, USA, 300–338.
- [26] James Prather, Brent N. Reeves, Juho Leinonen, Stephen MacNeil, Arisoa S Randrianasolo, Brett A. Becker, Bailey Kimmel, Jared Wright, and Ben Briggs. 2024. The Widening Gap: The Benefits and Harms of Generative AI for Novice Programmers. In *Proceedings of the 2024 ACM Conference on International Computing Education Research - Volume 1*. ACM, Melbourne VIC Australia, 469–486.
- [27] Fujiko Robledo Yamamoto, Lecia Barker, and Amy Volda. 2023. CIsing Up Service Learning: A Systematic Review of Service Learning Experiences in Computer and Information Science. *ACM Transactions on Computing Education* 23, 3 (Sept. 2023), 1–56. <https://doi.org/10.1145/3610776>
- [28] Jean Salac, Alannah Oleson, Lena Armstrong, Audrey Le Meur, and Amy J. Ko. 2023. Funds of Knowledge Used by Adolescents of Color in Scaffolded Sensemaking around Algorithmic Fairness. In *Proceedings of the 2023 ACM Conference on International Computing Education Research V.1*. ACM, Chicago IL USA, 191–205.
- [29] Sreecharan Sankaranarayanan, Siddharth Reddy Kandimalla, Christopher A. Bogart, R. Charles Murray, Michael Hilton, Majd F. Sakr, and Carolyn P. Rose. 2022. Collaborative Programming for Work-Relevant Learning: Comparing Programming Practice With Example-Based Reflection for Student Learning and Transfer Task Performance. *IEEE Transactions on Learning Technologies* 15, 5 (Oct. 2022), 594–604. <https://doi.org/10.1109/TLT.2022.3169121>
- [30] Sreecharan Sankaranarayanan, Siddharth Reddy Kandimalla, Mengxin Cao, Ignacio Maronna, Haokang An, Chris Bogart, R. Charles Murray, Michael Hilton, Majd Sakr, and Carolyn Penstein Rosé. 2020. Designing for Learning during Collaborative Projects Online: Tools and Takeaways. *Information and Learning Sciences* 121, 7/8 (June 2020), 569–577. <https://doi.org/10.1108/ILS-04-2020-0095>
- [31] Sreecharan Sankaranarayanan, Lanmingqi Ma, Siddharth Reddy Kandimalla, Ihor Markevych, Huy Nguyen, Charles Murray, Christopher Bogart, Michael Hilton, Majd Sakr, and Carolyn Penstein Rosé. 2022. Collaborative Reflection "in the Flow" of Programming: Designing Effective Collaborative Learning Activities in Advanced Computer Science Contexts. *CSC2022 Proceedings* (2022), 67–74.
- [32] Jingjing Shao, Yunshan Chen, Xiaoyang Wei, Xiaoran Li, and Yanyan Li. 2023. Effects of Regulated Learning Scaffolding on Regulation Strategies and Academic Performance: A Meta-Analysis. *Frontiers in Psychology* 14 (March 2023), 1110086.
- [33] Lev Tankelevitch, Viktor Kewenig, Auste Simkute, Ava Elizabeth Scott, Advait Sarkar, Abigail Sellen, and Sean Rintel. 2024. The Metacognitive Demands and Opportunities of Generative AI. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*. ACM, Honolulu HI USA, 1–24.
- [34] Maria Theobald. 2021. Self-Regulated Learning Training Programs Enhance University Students' Academic Performance, Self-Regulated Learning Strategies, and Motivation: A Meta-Analysis. *Contemporary Educational Psychology* 66 (July 2021), 101976. <https://doi.org/10.1016/j.cedpsych.2021.101976>
- [35] Tammy VanDeGrift. 2022. Post-Exam Videos for Assessment in Computing Courses: See and Hear Students' Thinking. In *Proceedings of the 53rd ACM Technical Symposium on Computer Science Education - Volume 1 (SIGCSE 2022)*. Vol. 1. Association for Computing Machinery, New York, NY, USA, 230–236.
- [36] Tammy VanDeGrift. 2024. Giving Voice to Problem-Solving: Hearing Students' Techniques in Video Reflections. In *2024 ASEE Annual Conference & Exposition*.
- [37] Xinghua Wang, Ingo Kollar, and Karsten Stegmann. 2017. Adaptable Scripting to Foster Regulation Processes and Skills in Computer-Supported Collaborative Learning. *International Journal of Computer-Supported Collaborative Learning* 12, 2 (June 2017), 153–172. <https://doi.org/10.1007/s11412-017-9254-x>
- [38] Benjamin Xie, Jared Ordona Lim, Paul K.D. Pham, Min Li, and Amy J. Ko. 2023. Developing Novice Programmers' Self-Regulation Skills with Code Replays. In *Proceedings of the 2023 ACM Conference on International Computing Education Research V.1*. ACM, Chicago IL USA, 298–313.
- [39] Xuesong Zhai, Xiaoyan Chu, Minjuan Wang, Chin-Chung Tsai, Jyh-Chong Liang, and Jonathan Michael Spector. 2024. A Systematic Review of Stimulated Recall (SR) in Educational Research from 2012 to 2022. *Humanities and Social Sciences Communications* 11, 1 (April 2024), 489.